BitX Agent Infrastructure

Executive Summary

The BitX platform operates as a **multi-tier, locally-hosted AI agent system** that requires no external LLM API dependencies. The infrastructure combines a Fastify-based Agent Hub (TypeScript/React/SQLite) with a legacy Product Hub (Spring Boot/Angular/PostgreSQL), orchestrating 26 specialized AI agents through a local LM Studio runtime.

This document provides a complete infrastructure reference covering deployment topology, service architecture, database design, network configuration, security controls, and operational procedures — everything required to deploy, operate, and maintain the BitX platform in development or production environments.

Key infrastructure characteristics:

- **Fully local LLM inference** — no cloud API calls, complete data sovereignty
- **Dual-platform architecture** — modern Agent Hub + legacy Product Hub
- **SQLite-backed persistence** — zero external database dependencies for the agent runtime
- **Docker-orchestrated services** — PostgreSQL, Neo4j, Valkey, Meilisearch for the legacy hub
- **Single-command startup** — `start.sh` brings up the entire ecosystem

---

1. Deployment Topology

1.1 System Context

```
+-----------------------------------------------------------------+
|                        BitX Platform                            |
|                                                                 |
|  +---------------------------+  +----------------------------+  |
|  |      Agents Hub           |  |      Product Hub           |  |
|  |  (Fastify + React)        |  |  (Spring Boot + Angular)   |  |
|  |  Port 4001 (API)          |  |  Port 8081 (API)           |  |
|  |  Port 3000 (UI)           |  |  Port 4201 (UI)            |  |
|  +-------------+-------------+  +-------------+--------------+  |
|                |                              |                 |
|  +-------------+-------------+  +-------------+--------------+  |
|  |        SQLite             |  |   Docker Services          |  |
|  |  data/producthub.db       |  |  PostgreSQL  :5432         |  |
|  |  (Drizzle ORM + WAL)     |  |  Neo4j       :7474/:7687   |  |
|  +---------------------------+  |  Valkey      :6379         |  |
|                                 |  Meilisearch :7700         |  |
|  +---------------------------+  +----------------------------+  |
|  |      LM Studio            |                                 |
|  |  localhost:1234           |                                 |
|  |  Router: Ministral 8B     |                                 |
|  |  Worker: Qwen2.5 32B      |                                 |
|  +---------------------------+                                 |
+-----------------------------------------------------------------+
```

1.2 Service Port Map

Service	Port	Protocol	Description
Agents Hub Client	3000	HTTP	React 19 frontend (Vite dev server)
Agents Hub Server	4001	HTTP/WS	Fastify API + WebSocket

Product Hub Backend	8081	HTTP	Spring Boot 3.2.2 REST API
Product Hub UI	4201	HTTP	Angular 17 frontend
LM Studio	1234	HTTP	OpenAI-compatible inference server
PostgreSQL	5432	TCP	Product Hub database
Neo4j HTTP	7474	HTTP	Neo4j browser/console
Neo4j Bolt	7687	TCP	Neo4j query protocol
Valkey	6379	TCP	Redis-compatible cache
Meilisearch	7700	HTTP	Full-text search engine

1.3 Network Architecture

```
Browser (localhost:3000)
    |
    |   HTTP (API proxy)
    v
Vite Dev Server (:3000)
    |
    |   Proxy rules (13 path patterns)
    v
Fastify Server (:4001) ---------> LM Studio (:1234)
    |                                 |
    |   Drizzle ORM                   |   OpenAI-compatible API
    v                                 v
SQLite (WAL mode)             GPU Memory (24+ GiB)
    |                                 |
    |                         +-------+-------+
    |                         |               |
    |                    Router Model    Worker Model
    |                    Ministral 8B    Qwen2.5 32B
    |                    (4.91 GiB)     (19.85 GiB)
    v
data/producthub.db
```

---

2. Repository Structure

2.1 Root Layout

```
BitX-Wiki-Agents/
├── agents/                        # 26 Agent JSON definitions (original)
│   ├── product-manager.json
│   ├── process-analyst.json
│   ├── business-analyst.json
│   ├── ... (23 more)
│   └── release-manager.json
├── agents-hub/                    # Main platform (Fastify + React)
│   ├── server/                    # TypeScript backend
│   ├── client/                    # React 19 frontend
│   ├── e2e/                       # Playwright E2E tests
│   ├── data/                      # SQLite database + uploads
```

```
│   ├── .claude/skills/             # 7 Claude Code skills
│   └── .env                        # Configuration
├── agents-local/                   # Local agent runtime resources
│   ├── profiles/                   # 26 agent profiles (auto-converted)
│   ├── docs/                       # Platform documentation
│   └── schemas/                    # Profile validation schemas
├── product-hub/                    # Legacy enterprise platform
│   ├── product-hub-backend/        # Spring Boot (Java 17)
│   ├── product-hub-ui/             # Angular 17
│   └── docker/                     # Docker Compose configs
├── BitX-Wiki/                      # Reference implementation
├── start.sh                        # Full system startup script
└── LOCAL_AGENT_MIGRATION_MASTER_PLAN.md
```

2.2 Server Module Map

The Agents Hub backend (`agents-hub/server/src/`) is organized into 15 modules:

```
server/src/
├── index.ts                  # App entry — Fastify init, plugin registration
├── config.ts                 # Environment-based configuration (CONFIG object)
├── auth/                     # JWT authentication (login, refresh, verify)
│   ├── auth.service.ts       # Token generation + bcrypt password hashing
│   ├── auth.middleware.ts    # Bearer token extraction middleware
│   └── auth.routes.ts        # POST /auth/login, /auth/refresh
├── db/                       # Database layer
│   ├── index.ts              # Drizzle ORM + raw better-sqlite3 exports
│   ├── schema.ts             # 9 core tables (users, agents, sessions, etc.)
│   ├── migrate.ts            # CREATE TABLE migrations
│   └── seed.ts               # Populate with 26 agents + admin user
├── local-agent/              # Local LM Studio runtime (10 modules)
│   ├── llm-gateway.ts        # OpenAI-compatible LM Studio client
│   ├── llm-gateway.config.ts # Router/Worker model configuration
│   ├── orchestrator.ts       # 6-stage agent execution pipeline
│   ├── profile-loader.ts     # Load agent profiles from disk
│   ├── run-store.ts          # SQLite persistence for runs/steps/artifacts
│   ├── run-store.schema.ts   # 4 tables (agentRuns, agentSteps, etc.)
│   ├── tool-gateway.ts       # 12 tools with schema validation + path safety
│   ├── rag-store.ts          # BM25/TF-IDF search (no external vector DB)
│   ├── rag-store.schema.ts   # ragDocuments + ragSearchLog tables
│   ├── rag-ingest.ts         # Background document ingestion
│   ├── validators.ts         # 7 code-enforced validators
│   ├── scheduler.ts          # Priority queue with concurrency limits
│   ├── eval-harness.ts       # 11 eval cases across 5 categories
│   └── local-agent.routes.ts # 20 REST endpoints
├── agents/                   # Agent CRUD + lookup
├── assignments/              # User-to-agent assignment mapping
├── audit/                    # Audit event logging
├── middleware/                # Request/response middleware
├── n8n/                      # n8n workflow integration
├── pipelines/                # Agent pipeline orchestration
├── sessions/                 # Chat session management
├── settings/                 # User/system settings
├── uploads/                  # File upload handling
├── user-journeys/            # User journey endpoints
```

```
├── users/                    # User CRUD
└── ws/                       # WebSocket handler
```

---

3. Technology Stack

3.1 Agents Hub Stack

Layer	Technology	Version	Purpose
Runtime	Node.js	20+	JavaScript/TypeScript execution
Framework	Fastify	5	HTTP server + WebSocket
Language	TypeScript	5.x	Type-safe development
ORM	Drizzle ORM	0.45	SQL query builder + schema management
Database	SQLite	better-sqlite3	Zero-config embedded database
Frontend	React	19	UI component framework
State	Zustand	latest	Lightweight state management
Build	Vite	7.3	Frontend build + dev server
Styling	Tailwind CSS	4.1	Utility-first CSS framework
HTTP Client	Axios	latest	API communication
Routing	React Router	7.x	Client-side navigation
Auth	JWT	HS256	Token-based authentication
Testing	Vitest	latest	Unit tests (110 tests across 6 files)
E2E Testing	Playwright	latest	Browser automation tests
ID Generation	@paralleldrive/cuid2	latest	Collision-resistant unique IDs
Password	bcrypt	latest	Password hashing (12 rounds)

3.2 Product Hub Stack (Legacy)

Layer	Technology	Version	Purpose
Backend	Spring Boot	3.2.2	Enterprise Java framework
Language	Java	17	Backend language
Build	Maven	3.x	Dependency management
Database	PostgreSQL	15	Relational data store
Graph DB	Neo4j	5 Community	Relationship modeling
Cache	Valkey	7	Redis-compatible caching
Search	Meilisearch	1.6	Full-text search
Auth	Keycloak	latest	Identity management
Frontend	Angular	17	Enterprise UI framework

Container	Docker	Compose	Service orchestration

3.3 LM Studio Runtime

Component	Specification	Details
Server	LM Studio	localhost:1234, OpenAI-compatible API
Router Model	Ministral 8B Instruct 2410	q4_k_m quantization, 4.91 GiB VRAM, 16K context
Worker Model	Qwen 2.5 Coder 32B Instruct	Default quantization, 19.85 GiB VRAM, 8K context
API Protocol	OpenAI Chat Completions	`/v1/chat/completions`, `/v1/models`
CLI Tool	lms.exe	`~/.lmstudio/bin/lms.exe` for model management

---

4. Database Architecture

4.1 SQLite Configuration (Agents Hub)

The Agents Hub uses a single SQLite database with Write-Ahead Logging for concurrent read performance:

```
Database: data/producthub.db
Mode:     WAL (Write-Ahead Logging)
Foreign Keys: ON
ORM:      Drizzle (better-sqlite3 driver)
```

Initialization sequence (from `db/index.ts`):

1. Ensure `data/` directory exists
2. Open SQLite database at `CONFIG.databasePath`
3. Enable WAL journal mode for concurrent reads
4. Enable foreign key enforcement
5. Export both Drizzle ORM (`db`) and raw SQLite (`sqlite`) handles

4.2 Core Tables (9 tables)

Table	Purpose	Key Columns
`users`	User accounts	id, username, displayName, passwordHash, role (admin/user), isActive
`agents`	Agent definitions	id, name, description, systemPrompt, allowedTools[], model, isActive
`user_agent_assignments`	User-to-agent mapping	userId (FK), agentId (FK)
`sessions`	Chat sessions	id, sdkSessionId, userId (FK), agentId (FK), title, totalCostUsd
`messages`	Chat history	id, sessionId (FK), role, content, metadata (JSON)
`audit_log`	Action tracking	userId (FK), agentId (FK), action, details, ipAddress
`refresh_tokens`	Token blacklist	userId (FK), tokenHash, expiresAt, isRevoked
`pipelines`	Agent workflows	id, name, steps (JSON), isDefault, createdBy (FK)
`pipeline_runs`	Workflow executions	pipelineId (FK), status, initialPrompt, inputType

4.3 Local Agent Tables (6 tables)

Run persistence (`run-store.schema.ts`):

Table	Purpose	Key Columns
`agent_runs`	Execution records	id, profileId, userId, agentId, tenantId, status (12 states), routerModel, workerModel, initialPrompt, finalOutput, totalTokens, totalDurationMs
`agent_steps`	Individual pipeline steps	runId (FK), stepOrder, stage (9 types), model, role, toolName, toolInput/Output (JSON), tokensUsed, durationMs
`agent_artifacts`	Generated deliverables	runId (FK), stepId (FK), type (11 categories), name, content, filePath, metadata (JSON)
`agent_approvals`	Human review gates	runId (FK), stepId (FK), requiredBy, reason, status (pending/approved/rejected), reviewerId

RAG persistence (`rag-store.schema.ts`):

Table	Purpose	Key Columns
`rag_documents`	Indexed document chunks	id, source, sourceType, title, content, contentHash (SHA-256), embedding (TF-IDF JSON), tenantId
`rag_search_log`	Search audit trail	query, runId, resultCount, durationMs, tenantId

4.4 Run Status State Machine

```
queued --> intake --> inventory --> context_gathering --> planning
                                                            |
                                                            v
completed <-- finalizing <-- approval <-- validation <-- tool_loop
    |
    +-- failed (from any stage)
    +-- cancelled (from queued or active)
```

4.5 Docker Database Services (Product Hub)

```
# docker/docker-compose.dev.yml

PostgreSQL 15 (Alpine):
  Port:     5432
  Database: producthub
  User:     postgres / password

Neo4j 5 Community:
  Ports:    7474 (HTTP browser), 7687 (Bolt protocol)
  Auth:     neo4j / password

Valkey 7 (Alpine):
  Port:     6379
  Purpose:  Redis-compatible session cache

Meilisearch 1.6:
  Port:     7700
  Key:      masterKey
  Purpose:  Full-text document search
```

---

5. The 26 Agents

5.1 Agent Inventory

Agents are organized into 5 SDLC phases plus an orchestrator role:

Phase	Agent	Prefix	Model Role	JSON Definition
Orchestrator	Product Manager	PM-	Router	`product-manager.json`
Discover	Process Analyst	PA-	Worker	`process-analyst.json`
	Business Analyst	BA-	Worker	`business-analyst.json`
	User Journey Agent	UJ-	Worker	`user-journey-agent.json`
Design	System Analyst	SA-	Worker	`system-analyst.json`
	Solutions Architect	ARCH-	Worker	`solutions-architect.json`
	UX Designer	UX-	Worker	`ux-designer.json`
	Technical Writer	DOC-	Worker	`technical-writer.json`
Build	Backend Developer	BE-	Worker	`backend-developer.json`
	Frontend Developer	FE-	Worker	`frontend-developer.json`
	Full Stack Developer	FS-	Worker	`full-stack-developer.json`
	Database Admin	DBA-	Worker	`database-admin.json`
	Data Engineer	DATA-	Worker	`data-engineer.json`
	DevOps Engineer	DEVOPS-	Worker	`devops-engineer.json`
Test	QA Lead	QA-	Worker	`qa-lead.json`
	Unit Tester	QA-UNIT-	Worker	`unit-tester.json`
	Integration Tester	QA-INT-	Worker	`integration-tester.json`
	Functional Tester	QA-FUNC-	Worker	`functional-tester.json`
	Regression Tester	QA-REG-	Worker	`regression-tester.json`
	Performance Engineer	QA-PERF-	Worker	`performance-engineer.json`
	Security Analyst	SEC-	Worker	`security-analyst.json`
	Accessibility Tester	QA-A11Y-	Worker	`accessibility-usability-tester.json`
	Cross-Platform Tester	QA-COMPAT-	Worker	`cross-platform-tester.json`
	BA Validation Tester	QA-BV-	Worker	`ba-validation-tester.json`
	UAT Coordinator	UAT-	Worker	`uat-coordinator.json`
Deploy	Release Manager	REL-	Worker	`release-manager.json`

5.2 Dual-Source Agent Loading

The Profile Loader supports two agent definition sources:

```
Source 1: agents-local/profiles/ (26 directories)
  - YAML/JSON profile files
  - Pre-structured with allowedTools, forbiddenPaths, validators
  - Directly loaded by profile-loader.ts

Source 2: agents/ (26 JSON files)
  - Original Claude agent definitions
  - Auto-converted at startup by convertClaudeAgent()
  - Maps Claude tool names to local equivalents via mapClaudeToolsToLocal()
```

5.3 Agent Profile Structure

Each profile defines operational boundaries for its agent:

Field	Type	Purpose
`id`	string	Unique identifier (e.g., `product_manager`)
`name`	string	Display name (e.g., `Product Manager`)
`prefix`	string	Artifact prefix (e.g., `PM-`)
`phase`	string	SDLC phase assignment
`modelRole`	`router` or `worker`	Which LLM model to use
`purpose`	string	Role description for system prompt
`systemPrompt`	string	Full system prompt (auto-generated from profile)
`allowedTools`	string[]	Whitelisted tool names
`forbiddenTools`	string[]	Blacklisted tool names
`allowedPaths`	string[]	Permitted file system paths
`forbiddenPaths`	string[]	Blocked paths (`.env`, `node_modules/`, `.git/`)
`approvalRequired`	boolean	Whether runs need human approval
`skills`	Skill[]	Slash commands the agent can execute
`receives`	string[]	What the agent expects from upstream agents
`produces`	string[]	What the agent delivers to downstream agents
`boundaries`	object	Always/Ask First/Never behavioral rules

5.4 Agent Handoff Chain

```
Discover Phase          Design Phase            Build Phase           Test Phase
+-------+              +-------+               +-------+             +-------+
| PA-   |---+          | SA-   |---+           | BE-   |---+         | QA-   |
| BA-   |   +--------->| ARCH- |   +---------->| FE-   |   +-------->| SEC-  |
| UJ-   |---+          | UX-   |---+           | FS-   |---+         | REL-  |
+-------+              | DOC-  |               | DBA-  |             +-------+
                       +-------+               | DATA- |
                                               | DEVOPS|
                                               +-------+
```

```
            PM- (Product Manager) orchestrates all phases
```

---

6. LLM Runtime Infrastructure

6.1 LM Studio Configuration

LM Studio runs as a local HTTP server exposing an OpenAI-compatible API:

Parameter	Value	Source
Base URL	`http://localhost:1234`	`LM_STUDIO_BASE_URL` env var
API Key	`lm-studio`	`LM_STUDIO_API_KEY` env var
API Endpoints	`/v1/chat/completions`, `/v1/models`	OpenAI-compatible
CLI Tool	`~/.lmstudio/bin/lms.exe`	Model loading/management

6.2 Dual-Model Architecture

```
                  +---------------------------+
                  |        LM Studio          |
                  |     localhost:1234         |
                  +---------------------------+
                  |                           |
          +------+------+           +--------+--------+
          | Router Model|           |  Worker Model   |
          +-------------+           +-----------------+
          | Ministral   |           | Qwen 2.5 Coder  |
          | 8B Instruct |           | 32B Instruct    |
          | 2410        |           |                 |
          +-------------+           +-----------------+
          | Quant: q4_k_m|          | Quant: default  |
          | VRAM: 4.91 GiB|         | VRAM: 19.85 GiB |
          | Context: 16K |          | Context: 8K     |
          +-------------+           +-----------------+
          | Temp: 0.3   |           | Temp: 0.2       |
          | MaxTokens:  |           | MaxTokens:      |
          |   4,096     |           |   8,192         |
          | Timeout:    |           | Timeout:        |
          |   30s       |           |   120s          |
          | Retries: 2  |           | Retries: 1      |
          +-------------+           +-----------------+
          |                         |
          | Used for:               | Used for:
          | - Profile routing       | - Code generation
          | - Task classification   | - Analysis
          | - Quick decisions       | - Complex reasoning
          +-------------------------+-----------------+
```

6.3 Model Configuration (from `llm-gateway.config.ts`)

```
Router Model:
  ID:             ministral-8b-instruct-2410@q4_k_m:2
  Role:           router
```

```
  Max Tokens:     4,096
  Temperature:    0.3 (deterministic)
  Timeout:        30 seconds
  Retries:        2
  Context Window: 32,768 (configured), 16K (actual hardware limit)

Worker Model:
  ID:             qwen2.5-coder-32b-instruct:2
  Role:           worker
  Max Tokens:     8,192
  Temperature:    0.2 (highly deterministic)
  Timeout:        120 seconds
  Retries:        1
  Context Window: 131,072 (configured), 8K (actual hardware limit)
```

6.4 Model Discovery and Health

The LLM Gateway performs automatic model management:

1. **Discovery** — queries `GET /v1/models` to list all loaded models
2. **Auto-bind** — matches discovered models to router/worker roles (skips if .env models found)
3. **Health check** — every 60 seconds, verifies models are responsive
4. **Fallback** — if enabled, attempts alternative model on primary failure

6.5 Hardware Requirements

Component	Minimum	Recommended
GPU VRAM	24 GiB (both models)	32+ GiB
System RAM	16 GiB	32 GiB
CPU	8 cores	16+ cores
Storage	50 GiB (models + data)	100+ GiB SSD
GPU	NVIDIA RTX 3090 or equivalent	RTX 4090 / A6000

---

7. Environment Configuration

7.1 Environment Variables

The platform is configured through a single `.env` file in `agents-hub/`:

```
# ── Authentication ──────────────────────────────────
JWT_ACCESS_SECRET       # HS256 signing key for access tokens (15min expiry)
JWT_REFRESH_SECRET      # HS256 signing key for refresh tokens (7-day expiry)

# ── Database ────────────────────────────────────────
DATABASE_PATH           # SQLite file path (default: ./data/producthub.db)

# ── Server ──────────────────────────────────────────
SERVER_PORT             # HTTP listen port (default: 4000, actual: 4001)
CLIENT_URL              # CORS allowed origin (default: http://localhost:3000)

# ── Agent Paths ─────────────────────────────────────
AGENTS_DIR              # Path to 26 agent JSON definitions (default: ../agents)
AGENT_WORKING_DIR       # Working directory for tool execution (default: ../product-hub)
```

```
# ── LM Studio / Local Agent Runtime ──────────────────
LM_STUDIO_BASE_URL      # LM Studio server URL (default: http://localhost:1234)
LM_STUDIO_API_KEY       # API key (default: lm-studio)
LM_ROUTER_MODEL         # Router model ID with quantization suffix
LM_WORKER_MODEL         # Worker model ID with quantization suffix
USE_LOCAL_AGENT         # Enable local runtime (default: true)
```

7.2 Current Production Values

```
JWT_ACCESS_SECRET=change_me_to_a_random_64_char_string
JWT_REFRESH_SECRET=change_me_to_a_different_random_64_char_string
DATABASE_PATH=./data/producthub.db
SERVER_PORT=4001
CLIENT_URL=http://localhost:3000
AGENTS_DIR=../agents
AGENT_WORKING_DIR=../product-hub
LM_STUDIO_BASE_URL=http://localhost:1234
LM_STUDIO_API_KEY=lm-studio
LM_ROUTER_MODEL=ministral-8b-instruct-2410@q4_k_m:2
LM_WORKER_MODEL=qwen2.5-coder-32b-instruct:2
USE_LOCAL_AGENT=true
```

---

8. API Surface

8.1 Endpoint Inventory

The Fastify server exposes 20+ REST endpoints plus WebSocket:

Health and Configuration:

Method	Path	Auth	Purpose
GET	`/health`	None	Server heartbeat
GET	`/local-agent/health`	Optional	LLM + scheduler status
GET	`/local-agent/models`	Optional	Discovered and bound models
POST	`/local-agent/models/discover`	Optional	Trigger model re-discovery
GET	`/claude-status`	None	Claude CLI availability check
GET	`/tunnel-config`	None	Cloudflare tunnel discovery

Agent Profiles:

Method	Path	Auth	Purpose
GET	`/local-agent/profiles`	Optional	List all 26 profiles (summary)
GET	`/local-agent/profiles/:id`	Optional	Full profile details

Agent Runs:

Method	Path	Auth	Purpose

POST	`/local-agent/runs`	Optional	Submit new agent run
GET	`/local-agent/runs`	Optional	List runs (admin: all; user: own)
GET	`/local-agent/runs/:id`	Optional	Run details + steps + artifacts
GET	`/local-agent/runs/:id/steps`	Optional	Step-by-step breakdown
GET	`/local-agent/runs/:id/artifacts`	Optional	Generated deliverables

Approvals:

Method	Path	Auth	Purpose
GET	`/local-agent/runs/:id/approvals`	Optional	Pending approval requests
POST	`/local-agent/approvals/:id/resolve`	Optional	Approve or reject

Queue Management:

Method	Path	Auth	Purpose
GET	`/local-agent/queue`	Optional	Queue depth + active runs
DELETE	`/local-agent/queue/:runId`	Optional	Cancel queued run

Tools and Validators:

Method	Path	Auth	Purpose
GET	`/local-agent/tools`	Optional	List 12 available tools
GET	`/local-agent/validators`	Optional	List 7 active validators
POST	`/local-agent/route`	Optional	Profile selection for prompt

Evaluation:

Method	Path	Auth	Purpose
GET	`/local-agent/eval/cases`	Optional	List 11 eval cases
POST	`/local-agent/eval/run`	Optional	Run full benchmark suite
POST	`/local-agent/eval/run-single`	Optional	Run single eval case

Authentication:

Method	Path	Auth	Purpose
POST	`/auth/login`	None	Username/password → JWT tokens
POST	`/auth/refresh`	None	Refresh token → new access token

8.2 WebSocket

```
Endpoint:  ws://localhost:4001/ws
Purpose:   Real-time run progress updates
Protocol:  JSON messages with { type, runId, stage, progress, data }
```

8.3 API Proxy Configuration (Vite)

The React frontend proxies 13 path patterns to the Fastify backend:

```
/auth         → http://127.0.0.1:4001
/me           → http://127.0.0.1:4001
/agents       → http://127.0.0.1:4001
/sessions     → http://127.0.0.1:4001
/admin        → http://127.0.0.1:4001
/api          → http://127.0.0.1:4001
/health       → http://127.0.0.1:4001
/pipelines    → http://127.0.0.1:4001
/pipeline-runs→ http://127.0.0.1:4001
/upload       → http://127.0.0.1:4001
/uploads      → http://127.0.0.1:4001
/settings     → http://127.0.0.1:4001
/about/agents → http://127.0.0.1:4001
/ws           → ws://127.0.0.1:4001 (WebSocket upgrade)
```

---

9. Security Architecture

9.1 Authentication Flow

```
1. Login:  POST /auth/login  { username, password }
              |
              v
   Verify password with bcrypt (12 rounds)
              |
              v
   Generate accessToken (HS256, 15min expiry)
   Generate refreshToken (HS256, 7-day expiry)
   Store refresh token hash in refresh_tokens table
              |
              v
   Return { accessToken, refreshToken, user }

2. API Call:  Authorization: Bearer <accessToken>
              |
              v
   Verify signature with JWT_ACCESS_SECRET
   Attach decoded { userId, role } to request.user
              |
              v
   Role check: admin vs user

3. Refresh:  POST /auth/refresh  { refreshToken }
              |
              v
   Verify signature + check not revoked in DB
              |
              v
   Issue new accessToken (old refresh token stays valid)
```

9.2 Role-Based Access Control

Role	Capabilities
admin	Full access: user management, agent CRUD, all runs, audit log
user	Own runs only, assigned agents only, no admin panel
unauthenticated	Local agent routes only (optional auth for local dev)

9.3 Agent Execution Safety

Control	Implementation	Source
Tool allowlist	Each profile specifies `allowedTools[]` — only listed tools can execute	`profile-loader.ts`
Tool denylist	`forbiddenTools[]` blocks specific tools per agent	`profile-loader.ts`
Path safety	`forbiddenPaths` blocks access to `.env`, `node_modules/`, `.git/`, `data/`	`tool-gateway.ts`
Traversal detection	Rejects paths containing `..` or absolute paths outside working directory	`tool-gateway.ts`
Schema validation	Every tool input validated against Zod schema before execution	`tool-gateway.ts`
Diff threshold	Maximum allowed changed lines per run (profile-configurable)	`validators.ts`
Blocked patterns	Regex patterns that must not appear in outputs (e.g., secrets)	`validators.ts`
Required tests	Some profiles require test execution before finalizing	`validators.ts`
Human approval	`approvalRequired` profiles pause for manual review	`validators.ts`

9.4 Network Security

Control	Configuration
CORS	Origin whitelist: `CLIENT_URL` + `trycloudflare.com`
Rate limiting	1,000 requests/minute per IP (Fastify plugin)
File upload	10 MB maximum via `@fastify/multipart`
WebSocket	Upgrade on `/ws` path only
Binding	Server listens on `0.0.0.0` (all interfaces)

---

10. Scheduler and Concurrency

10.1 Scheduler Configuration

Parameter	Default	Purpose
`maxConcurrentRuns`	2	Maximum parallel agent executions
`maxQueueSize`	50	Maximum pending runs in queue
`maxRunsPerTenant`	3	Per-tenant concurrency limit
`queuePollIntervalMs`	2,000	Queue processing interval

10.2 Queue Processing

```
Incoming Run Request
        |
        v
  Queue full? (>50) ----Yes----> HTTP 429 "Queue is full"
        |
        No
        v
  Tenant limit? (>3) ---Yes----> HTTP 429 "Tenant limit reached"
        |
        No
        v
  Add to priority queue
  Sort by: priority ASC, createdAt ASC
        |
        v
  Active < maxConcurrent? --No--> Wait in queue (poll every 2s)
        |
        Yes
        v
  Dequeue + Execute via Orchestrator
        |
        v
  On completion: remove from active, trigger processQueue()
```

10.3 Priority Levels

Priority	Value	Use Case
Critical	1	System-initiated runs, eval benchmarks
High	5	Admin-triggered runs
Normal	10	Standard user requests (default)
Low	20	Background tasks, re-processing
Batch	50	Bulk operations, scheduled jobs

---

11. Startup Sequence

11.1 Full Platform Startup (`start.sh`)

```
Step 1: Prerequisites Check
  ├── node --version        (required)
  ├── npm --version         (required)
  ├── docker --version      (required for Product Hub databases)
  ├── java --version        (optional for Product Hub backend)
  └── claude --version      (optional for Claude CLI integration)

Step 2: Install Dependencies
  ├── cd agents-hub && npm install
  └── cd product-hub/product-hub-ui && npm install

Step 3: Environment Setup
  └── cp .env.example .env  (if .env missing)
```

```
Step 4: Database Initialization
  ├── Run Drizzle migrations (CREATE TABLE for 15 tables)
  └── Seed database (26 agents + admin user)

Step 5: Start Docker Services
  └── docker compose -f docker/docker-compose.dev.yml up -d
      ├── PostgreSQL  :5432
      ├── Neo4j       :7474/:7687
      ├── Valkey      :6379
      └── Meilisearch :7700

Step 6: Start Application Services
  ├── Agents Hub Server     (port 4001)
  ├── Agents Hub Client     (port 3000)
  ├── Product Hub Backend   (port 8081)
  └── Product Hub UI        (port 4201)

Step 7: Health Verification
  ├── curl http://localhost:4001/health
  └── curl http://localhost:4001/local-agent/health
```

11.2 Local Agent Runtime Initialization

When the Fastify server starts, `initLocalAgentRuntime()` executes:

```
1. Profile Loader
   ├── Scan agents-local/profiles/ for YAML/JSON files
   ├── Scan agents/ for Claude agent JSON definitions
   ├── Auto-convert Claude agents to local profile format
   └── Cache all 26 profiles in memory

2. LLM Gateway
   ├── Query LM Studio /v1/models for available models
   ├── Match discovered models to .env configuration
   ├── Bind router model (Ministral 8B)
   ├── Bind worker model (Qwen 2.5 Coder 32B)
   └── Start health check timer (60s interval)

3. RAG Store
   ├── Ensure rag_documents and rag_search_log tables exist
   ├── Run background ingestion of docs/, profiles/, agents/, schemas/
   └── Index all documents with TF-IDF vectors

4. Scheduler
   └── Start queue processing timer (2s interval)
```

---

12. Monitoring and Observability

12.1 Built-In Health Endpoints

`GET /local-agent/health` returns:

```
{
  "status": "healthy | degraded",
  "models": [
    { "id": "ministral-8b-instruct-2410@q4_k_m:2", "role": "router", "available": true },
```

```
    { "id": "qwen2.5-coder-32b-instruct:2", "role": "worker", "available": true }
  ],
  "profiles": 26,
  "queue": {
    "queueLength": 0,
    "activeRuns": 1,
    "maxConcurrent": 2
  },
  "timestamp": "2026-03-05T..."
}
```

12.2 Telemetry Data Points

Metric	Source Table	Signal
Token usage per step	`agent_steps.tokens_used`	Cost tracking, prompt optimization
Duration per step	`agent_steps.duration_ms`	Performance bottlenecks
Tool call patterns	`agent_steps.tool_name`	Agent efficiency
Run success rate	`agent_runs.status`	Profile quality
RAG search latency	`rag_search_log.duration_ms`	Index performance
RAG hit rate	`rag_search_log.result_count`	Knowledge coverage
Queue depth	Scheduler in-memory	Capacity planning
Model availability	Health check endpoint	Uptime monitoring

12.3 Logging

All runtime components emit structured console logs:

```
[Scheduler] Started with maxConcurrent=2, maxQueue=50
[Scheduler] Enqueued run clx8k... at position 1
[Scheduler] Starting run clx8k... (active: 1/2)
[Orchestrator] Starting run clx8k... with profile process_analyst
[Orchestrator] Stage intake completed (1,204ms)
[LLMGateway] chat() model=qwen2.5-coder-32b-instruct:2 tokens=2,847
[ToolGateway] Executing read_file for run clx8k...
[Validator] Running 3 validators for profile process_analyst
[Scheduler] Run clx8k... completed with status: completed
```

---

13. Testing Infrastructure

13.1 Test Pyramid

```
+---------------------------+
|     E2E Tests             |    Playwright (browser automation)
|     agents-hub/e2e/       |    Login, agent execution, artifacts
+---------------------------+
|     Eval Harness          |    11 cases across 5 categories
|     eval-harness.ts       |    BA, Engineering, Testing, Repair, Adversarial
+---------------------------+
|     Agent Integration     |    26/26 agents tested and passing
```

```
|     test-all-agents.sh    |    Full end-to-end per-agent runs
+---------------------------+
|     Unit Tests            |    Vitest, 110 tests across 6 files
|     __tests__/            |    Validators, RAG, tools, profiles
+---------------------------+
```

13.2 Unit Tests

Test File	Module	Test Count
`validators.test.ts`	7 validators	~25
`rag-store.test.ts`	BM25 search + indexing	~20
`tool-gateway.test.ts`	12 tools + path safety	~25
`profile-loader.test.ts`	Profile loading + conversion	~15
`orchestrator.test.ts`	Pipeline stages	~15
`scheduler.test.ts`	Queue + concurrency	~10

13.3 Eval Harness

11 automated quality benchmarks:

Category	Count	What It Measures
BA	2	User story generation, requirements analysis
Engineering	2	Endpoint creation, bug fixing
Testing	2	Test execution, test planning
Repair	1	Fix failing test from error output
Adversarial	4	`.env` access, path traversal, SQL injection, write from read-only

Pass threshold: weighted score >= 70%

13.4 Full Agent Test Results

All 26 agents tested via `test-all-agents.sh`:

- **Pass rate**: 26/26 (100%)
- **Duration range**: ~30–270 seconds per agent
- **Token range**: 6K–55K tokens per run

---

14. Production Deployment Guide

14.1 Development Deployment (Current)

Single machine running all services:

```
Machine: Windows Server 2025
GPU:     24+ GiB VRAM (NVIDIA)
RAM:     32 GiB
Storage: SSD with 100 GiB free

Running:
```

```
  - LM Studio (GPU inference)
  - Node.js (Agents Hub server + client)
  - Docker (PostgreSQL, Neo4j, Valkey, Meilisearch)
  - Java (Product Hub backend, optional)
```

14.2 Production Deployment (Recommended)

```
Server 1: Application Tier
  ├── Agents Hub Server (Fastify, port 4001)
  ├── Agents Hub Client (built React, served by nginx)
  ├── SQLite → migrate to PostgreSQL for multi-server
  ├── Process manager: PM2 or systemd
  └── Reverse proxy: nginx with TLS termination

Server 2: Legacy Platform
  ├── Product Hub Backend (Spring Boot, port 8081)
  ├── Product Hub UI (Angular, served by nginx)
  └── Connects to shared database services

Server 3: GPU Inference
  ├── LM Studio on 0.0.0.0:1234
  ├── GPU: NVIDIA A6000 or RTX 4090 (24+ GiB VRAM)
  ├── Router Model: Ministral 8B (4.91 GiB)
  └── Worker Model: Qwen 2.5 Coder 32B (19.85 GiB)

Server 4: Database Services
  ├── PostgreSQL 15 (managed, with replication)
  ├── Neo4j 5 (community or enterprise)
  ├── Valkey/Redis (or managed ElastiCache)
  └── Meilisearch (or managed search service)
```

14.3 Production Checklist

Category	Action	Status
Security	Replace default JWT secrets with 64-char random strings	Required
Security	Change default admin password (`admin123`)	Required
Security	Configure CORS origin to production domain	Required
Security	Enable HTTPS/TLS via reverse proxy	Required
Security	Audit `.env` — no secrets in source control	Required
Database	Migrate SQLite to PostgreSQL for multi-server	Recommended
Database	Configure automated backups	Required
Database	Set up database replication/failover	Recommended
Networking	Configure firewall rules (only expose 80/443)	Required
Networking	Set up load balancer for high availability	Recommended
Monitoring	Deploy health check monitoring (uptime)	Required
Monitoring	Configure log aggregation (ELK or Loki)	Recommended

Monitoring	Set up alerting (model offline, queue full)	Recommended
GPU	Monitor VRAM usage and inference latency	Required
GPU	Plan for model upgrades (larger context, newer models)	Planned
Process	Document runbook for restarts, scaling, emergencies	Required
Process	Establish backup/restore procedures	Required

---

15. Scaling Considerations

15.1 Current Limitations

Component	Limitation	Mitigation
SQLite	Single-writer, not suitable for multi-server	Migrate to PostgreSQL
Scheduler	Single-instance, in-memory queue	Add message queue (RabbitMQ/Kafka)
LM Studio	Single GPU server, sequential inference	Add GPU cluster or batch API
WebSocket	Single-server affinity	Use sticky sessions or Redis pub/sub
Concurrency	Max 2 parallel runs (hardware limited)	Scale GPU servers + increase limit

15.2 Scaling Roadmap

```
Phase 1 (Current): Single Machine
  - 2 concurrent agent runs
  - SQLite database
  - All services co-located

Phase 2: Separated Services
  - PostgreSQL replaces SQLite
  - LM Studio on dedicated GPU server
  - Application tier behind load balancer

Phase 3: Horizontal Scaling
  - Multiple application servers
  - Message queue for run distribution
  - GPU cluster with model sharding
  - CDN for static frontend assets

Phase 4: Enterprise
  - Kubernetes orchestration
  - Auto-scaling GPU nodes
  - Multi-region deployment
  - 99.9% SLA target
```

Cross-references: AI Engine Architecture (Engine internals) | Learning Methodology (RAG store and knowledge pipeline) | UI/UX Design (Frontend architecture) | Dashboards (Monitoring specifications) | DDA Process Analyst (Agent in action)